

IoT Messaging Protocols

INSISN – Integração de Sistemas e Serviços na Nuvem

António Barros

Departamento de Engenharia Informática

IoT Communication Requirements

- **Resource-Constrained Environments:** Microcontrollers often operate with limited RAM, CPU cycles, and power reserves.
- **Network Constraints:** Low bandwidth, high latency, variable link quality, and frequent packet loss.
- **Scalability Needs:** Systems must handle thousands to millions of concurrent end-nodes seamlessly.
- **Limitations of Traditional HTTP:**
 - ▶ High header overhead (hundreds of bytes per request).
 - ▶ Synchronous polling models drain battery life rapidly.
 - ▶ Strictly request-driven architecture; inefficient for event-driven telemetry.

IoT Messaging Protocols

Architectural Paradigms

Publish/Subscribe (Pub/Sub)

Events are published to a broker, which then distributes them to interested subscribers based on topic subscriptions.

- **Decoupled:** Publishers and subscribers do not interact directly.
- **Central Broker / Bus:** Manages message routing based on topics or data types.
- **Protocols:** MQTT, AMQP, DDS.
- **Use Cases:** Telemetry, sensor data streaming, and event-driven architectures.

Request/Response

Clients send requests to servers, which process and return responses. This model is synchronous and tightly coupled.

- **Direct/Point-to-Point:** Client sends a request, server returns a response.
- **RESTful Mapping:** Maps directly to HTTP-style verbs (GET, POST, PUT, DELETE).
- **Protocols:** CoAP.
- **Use Cases:** Ideal for configuration, control commands, and resource retrieval.

Protocol Comparison Matrix

Feature	MQTT	CoAP	AMQP	DDS
Transport	TCP	UDP	TCP	UDP / TCP
Architecture	Pub/Sub (Broker)	Req/Resp & Pub/Sub	Pub/Sub & Queues	Data-Centric Pub/Sub
Header Size	Small (~2 B)	Very Small (~4 B)	Medium / Large	Variable
Overhead	Low	Very Low	High	Medium
Latency	Low	Very Low	Medium	Extremely Low
Target Domain	Telemetry / Constraints	Constrained Devices	Enterprise Integration	Real-Time / Defense

Overhead and Performance Profile

- **Footprint vs Transmission Effort:**

- ▶ **CoAP:** Runs over UDP; eliminates TCP connection state overhead and handshake latency. Ideal for sleep-cycling devices.
- ▶ **MQTT:** Requires persistent TCP connection; low overhead per message after connection setup.
- ▶ **AMQP:** Feature-rich frame-based routing (queues, exchanges) increases parsing complexity and byte overhead.
- ▶ **DDS:** Peer-to-peer architecture removes broker bottlenecks, achieving ultra-low latency at the expense of higher discovery traffic.

Typical Protocol Implementations and Middleware

Protocol	Architecture	Prominent Brokers / Middleware
MQTT	Broker-based	Eclipse Mosquitto, EMQX, HiveMQ, AWS IoT Core
AMQP	Broker-based	RabbitMQ, Apache ActiveMQ, Apache Qpid, Azure Service Bus
CoAP	Client / Server (No Broker)	Eclipse Californium, Erbium (Contiki-NG), CoAPthon
DDS	Peer-to-Peer (Brokerless)	eProsima Fast DDS, RTI Connext, Cyclone DDS, OpenDDS

MQTT: Message Queuing Telemetry Transport

Introduction to MQTT: Key Core Properties

OASIS Open Standard: Originally developed by IBM and Arcom in 1999; now the *de facto* standard for lightweight telemetry.

Essential Protocol Characteristics

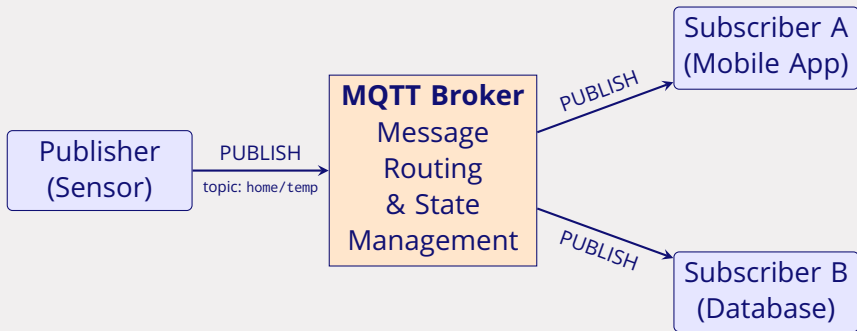
Binary-Agnostic: Payload contents are completely transparent to the broker (JSON, Protocol Buffers, CBOR, or raw binary).

Stateful Sessions: The broker maintains client session state, queuing messages for offline clients with active subscriptions.

Connection Keep-Alive: Periodic PINGREQ / PINGRESP heartbeats detect silent network drops without sending heavy payload data.

Flexible Transport: Operates natively over standard TCP/IP sockets, TLS-encrypted connections, or WebSockets (port 80/443).

MQTT Broker Architecture



MQTT shifts execution complexity from constrained endpoints to the central broker, leaving edge devices with minimal runtime overhead.

MQTT Ecosystem & Broker Implementations

The MQTT ecosystem spans from **lightweight brokers** for edge gateways to **distributed enterprise platforms** handling millions of concurrent connections.

Lightweight & Edge Brokers

Eclipse Mosquitto: Lightweight, open-source reference implementation (C). Ideal for edge gateways, small-scale deployments, and home automation.

Managed Cloud Platforms

AWS IoT Core / Azure IoT Hub: Fully managed cloud services using MQTT as the primary ingress interface.

Distributed Enterprise Brokers

EMQX: High-performance Erlang/OTP broker designed for massive horizontal scaling in industrial IoT.

HiveMQ: Enterprise-grade broker (Cloud/Self-hosted) tailored for automotive and manufacturing sectors.

VerneMQ: Distributed, high-availability broker focused on clustering and low-latency message delivery.

MQTT Topics & Wildcards

- **Topic Structure:** Hierarchical string separated by forward slashes (/).
- **Example:** factory/building1/sectionA/temperature

Wildcard Support

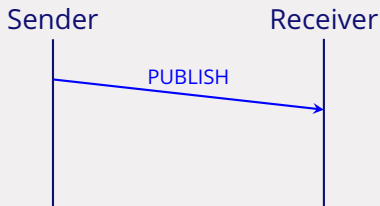
- + **Single-level wildcard:** Matches exactly one level in the topic hierarchy.
 - Example: factory/+/sectionA/temperature
- # **Multi-level wildcard:** Matches any number of trailing levels. Must be placed at the end.
 - Example: factory/building1/#

Quality of Service (QoS) Levels Overview

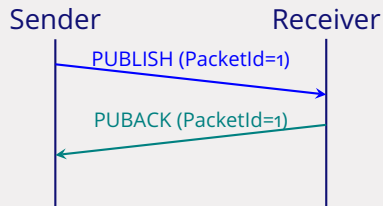
Level	Guarantee	Mechanism	Key Characteristics
QoS 0	At Most Once	No ACK	Best-effort delivery. Fire and forget; no retransmission.
QoS 1	At Least Once	PUBACK	Guaranteed delivery. Possible duplicate messages if ACKs are lost.
QoS 2	Exactly Once	4-Way Handshake	Guaranteed single delivery. Eliminates duplicates at the expense of higher network overhead.

QoS 0 & QoS 1 Packet Handshakes

QoS 0 (At Most Once)



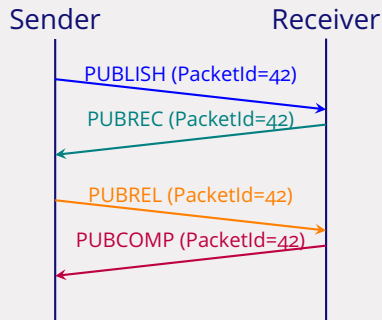
QoS 1 (At Least Once)



PUBACK (ACKNOWLEDGE): Acknowledgment packet sent by the receiver to confirm successful receipt of a QoS 1 message.

QoS 2: Four-Way Handshake Mechanics

QoS 2 (Exactly Once)



QoS 2: Four-Way Handshake

- **PUBLISH**: Sender transmits the message with a unique Packet Identifier.
- **PUBREC (RECEIVED)**: Receiver acknowledges receipt of the PUBLISH message and indicates it is ready for the next step.
- **PUBREL (RELEASE)**: Sender indicates it is ready to complete the transaction.
- **PUBCOMP (COMPLETE)**: Receiver confirms the completion of the QoS 2 message exchange, ensuring exactly-once delivery.

Retained Messages and Initial State

- **Problem:** New subscribers normally wait until the *next* PUBLISH event to receive data, leaving them in an unknown state upon startup.
- **How the Retain Flag Works:** When a message is published with `retain=true`, the broker delivers it immediately and stores the **last known good message** for that topic.

Key Operational Characteristics

- **One Message per Topic:** The broker stores only *one* retained message per specific topic string. Publishing a new retained message overwrites the previous one.
- **Clearing a Retained Message:** Publish a payload of length zero (empty payload) with `retain=true` to purge it from the broker's memory.

Retained Messages and Initial State (example)

Python Example: Publishing Device State

```
import paho.mqtt.client as mqtt

client = mqtt.Client(client_id="Temperature_Sensor_01")
client.connect("broker.hivemq.com", 1883)

# Publish current status as a retained message
client.publish("factory/sensor1/status", payload="OPERATIONAL", qos=1, retain=True)
```

Session Management: Client and Broker State

Identity and State: Every MQTT session is bound to a unique ClientID. The broker maintains state across disconnections based on connection flags.

What Constitutes an MQTT Session State?

- **Subscriptions:** All active topic filters subscribed to by the client.
- **In-Flight Messages:** Unacknowledged QoS 1 and QoS 2 messages being transmitted.
- **Message Queue:** Undelivered QoS 1 and QoS 2 messages published while the client was offline.

Session Management: Client and Broker State

Specification Evolution: MQTT 3.1.1 vs. MQTT 5.0

Clean Session (v3.1.1): Binary choice (True/False). Persistent sessions remained in broker memory indefinitely until manual purging.

Clean Start + Expiry (v5.0): Decouples connection cleanup from session lifetime using Session Expiry Interval (seconds) to prevent broker memory leaks.

Persistent Sessions: Feature Comparison

Feature	Clean Session = True	Persistent Session (Clean Session = False)
Lifecycle	Transient; destroyed upon client disconnection.	Persistent; retained by broker during of-line periods.
Subscriptions	Purged immediately on disconnect.	Stored by broker; automatically active upon reconnect.
In-Flight QoS	Undelivered QoS 1/2 messages are discarded.	QoS 1/2 messages are queued and retransmitted.
Target Domain	Ephemeral dashboards, real-time monitors.	Battery-powered edge nodes, unstable cellular links.

When a persistent client reconnects, the broker responds with a CONNACK packet containing Session Present = 1 and flushes all queued QoS 1/2 messages in sequence.

Advanced Features: LWT and Keep Alive

Keep Alive and Heartbeats

- Configured via KeepAlive timer in seconds during CONNECT.
- If no control packets are sent within $1.5 \times \text{KeepAlive}$, client sends PINGREQ.
- Broker responds with PINGRESP.
- Detects half-open TCP connections without sending heavy data.

Last Will and Testament (LWT)

- Registered with the broker during the initial CONNECT frame.
- **Trigger:** If the client disconnects *ungracefully* (power loss, signal drop), the broker automatically publishes the LWT message.
- **Clean Disconnect:** A graceful DISCONNECT packet cancels the LWT.

Advanced Features: LWT and Keep Alive

Python Configuration: LWT + Persistent Session

```
client = mqtt.Client(client_id="Gateway_Node_01", clean_session=False)

# Register Will prior to connecting
client.will_set("gateways/node01/health", payload="OFFLINE (Unexpected)", qos=1, retain=True)
client.connect("broker.hivemq.com", port=1883, keepalive=30)
```

Example: Python Paho MQTT Client Initialization

```
import paho.mqtt.client as mqtt

def on_connect(client, userdata, flags, rc):
    print(f"Connected with result code {rc}")
    client.subscribe("factory/building1/#", qos=1)

client = mqtt.Client(client_id="Gateway_01", clean_session=False)

# Register Last Will and Testament
client.will_set("factory/gateway/status", payload="OFFLINE", qos=1, retain=True)

client.on_connect = on_connect
client.connect("broker.hivemq.com", 1883, keepalive=60)

# Publish retained initial state
client.publish("factory/gateway/status", payload="ONLINE", qos=1, retain=True)

client.loop_start()
```